AGENCY PARTNER INTERACTIVE

Go-live pipeline

Briefing for DevOps

What actually ships today · 12 October 2026

This note describes the delivery path in the running application. It is written so a DevOps engineer can see what runs, what it depends on, and what is not wired up yet.

> Go-live is two separate deliveries after one Stripe payment. GitHub stores the site files. Vercel is what puts a managed-hosting site on the internet. A GitHub push does not deploy anything.

1. The two packages

The customer chooses a package on the quote. Checkout is created by the app. DevOps does not start a deploy by hand.

Package	Stripe checkout	Customer receives	What goes online
Self-hosting	One-time payment	ZIP download	Nothing. They host it.
Managed hosting	Creation fee plus a monthly subscription	A live URL	This app uploads static files straight to Vercel.

Both packages also commit the site into one shared GitHub repository, AI-WEBSITE-BUILDER-WEBSITE, under clients/<business-slug>-<last-6-of-project-id>/.

2. Sequence

```
Customer pays on Stripe Checkout
        |
        +-- Stripe webhook   POST /api/stripe/webhook     (primary)
        |
        +-- Customer lands on
            /app/project/:id/deployment?session_id=...     (backup)
        |
        v
Mark Payment = PAID, Project = PAID
        |
        +-- Commit full site to GitHub          (both packages)
        +-- Queue HubSpot deal update           (does not block delivery)
        |
        +-- Managed hosting only:
              upload public files to Vercel
              wait until Vercel says READY
              save the *.vercel.app URL
              Project = LIVE
              email the account owner
```

The customer never downloads GitHub, never logs into Vercel, and never picks a domain. They click Make it live, or the app does it for them.

3. Step by step

3.1 Checkout

POST /api/checkout/website recomputes the quote on the server, then opens a Stripe Checkout Session.

- Self-hosting uses mode payment and one line item.
- Managed hosting uses mode subscription: creation fee plus the monthly price.
- Success URL returns to /app/project/{id}/deployment?checkout=success&session_id={CHECKOUT_SESSION_ID}.
- A Payment row is written as PENDING before the customer leaves the app.

Price IDs, when set: STRIPE_SELF_HOSTING_PRICE_ID, STRIPE_MANAGED_CREATION_PRICE_ID, STRIPE_MANAGED_MONTHLY_PRICE_ID. If a price ID is empty, the app sends an inline price from the quote.

3.2 What counts as paid

Delivery unlocks only when Stripe payment_status is paid or no_payment_required. A Checkout Session that is merely complete is ignored. Delayed payment methods can finish the session before the funds settle. Those settle later on checkout.session.async_payment_succeeded.

Two callers run the same fulfillment function. Both are safe to run twice, keyed on the Stripe session id.

- Webhook events: checkout.session.completed and checkout.session.async_payment_succeeded.
- Delivery page: on return, the app retrieves that session from Stripe and fulfills it if the webhook has not landed. If session_id is gone from the URL, it retries the latest PENDING website payment.

The webhook signature is verified with STRIPE_WEBHOOK_SECRET. In production a missing secret rejects the event.

3.3 Fulfillment

- Payment.status becomes PAID. The amount stored is session.amount_total, what Stripe charged.
- Project.status becomes PAID, unless it is already DEPLOYING, LIVE, or EXPORTED.
- A HubSpot deal update is queued. A HubSpot failure does not stop delivery.

3.4 GitHub is the archive

Right after payment, the app builds the static site in memory and commits it with the GitHub Git Data API (blobs, tree, commit, then update the default branch). It does not open a pull request and it does not create a repository per customer. A GitHub push does not trigger Vercel.

- Repo: GITHUB_SITES_REPO (owner/repo), or ${GITHUB_ORG}/AI-WEBSITE-BUILDER-WEBSITE if that is unset.
- Token: GITHUB_TOKEN with write access on that repo.
- Folder: clients/bright-smile-ab12cd/ — slug of the business name plus the last 6 characters of the project id.
- Commit message: chore: sync <folder> (<business name>).
- If the repo has zero commits, the app writes a README first so later commits can land.
- Stored on the project: githubRepoUrl, githubCommitSha, githubSyncStatus = SYNCED or FAILED.

A GitHub failure does not block Vercel. The payment still stands. Later edits to a paid site commit again to that same folder. Unpaid drafts stay only in the database.

3.5 Managed hosting is a direct Vercel upload

This runs only if there is a PAID payment with package MANAGED_HOSTING, and the project is not already LIVE or DEPLOYING.

It starts in two places:

- The webhook schedules it after the HTTP response (after()), so Stripe is not held open for the upload.
- The delivery page, once it sees the project is paid and not live, calls POST /api/projects/{id}/deploy. That is the retry if the webhook publish fails. The customer can also press Make it live or Update live site.

DEPLOYMENT_TOKEN must be set, or the publish returns HTTP 503 (HOSTING_NOT_CONFIGURED). Optional team scope: DEPLOYMENT_TEAM_ID. The app also accepts VERCEL_TEAM_ID or VERCEL_ORG_ID. DEPLOYMENT_PROVIDER defaults to vercel.

What the app sends to Vercel:

- Build the same file set as the ZIP.
- Keep only visitor files: HTML, images, fonts, CSS, JS, templates/, uploads/, and vercel.json. blueprint.json, site.json, README.md, and package.json stay in GitHub and the ZIP. They are not uploaded.
- SHA1 each file and POST it to https://api.vercel.com/v2/files, 8 files at a time.
- POST https://api.vercel.com/v13/deployments with target production, framework null (static, no build). The Vercel project name is the same slug as the GitHub folder.
- Poll that deployment every 2 seconds for up to 90 seconds until readyState is READY.
- Store the URL Vercel returns. That is a *.vercel.app host. The app prefixes https:// and does not invent a custom domain.

vercel.json in the bundle is only { "cleanUrls": true, "trailingSlash": false }.

The webhook route and the deploy route allow up to 300 seconds (maxDuration). The Vercel wait itself gives up at 90 seconds.

3.6 Success and failure

Moment	Project status	Deployment row
Publish starts	DEPLOYING	BUILD_STARTED, logs appended
Vercel READY	LIVE	DEPLOYED, liveUrl saved
Upload, API error, or timeout	back to PAID	FAILED, error in logs

On success the owner is emailed the live URL (Resend, if email is configured). HubSpot is queued again so the deal points at the live URL instead of the in-app preview. The customer sees the log on the Deliver page: Starting, Building, Live, or Couldn't go live.

3.7 Self-hosting never calls Vercel

GET /api/projects/{id}/download is allowed after any paid website package. It zips the full file set (HTML, README, site.json, blueprint.json, package.json, .env.example, vercel.json). If the project is still PAID, QUOTED, or GENERATED, it is marked EXPORTED and the owner is emailed. A site that is already LIVE is not downgraded by a download.

4. What DevOps must have in place

Piece	Needed	Notes
Public app URL	Yes	Stripe success and cancel URLs are built from it.
STRIPE_SECRET_KEY	Yes	Checkout.
STRIPE_WEBHOOK_SECRET	Yes in production	Endpoint: POST /api/stripe/webhook.
Stripe webhook events	Two events	checkout.session.completed and checkout.session.async_payment_succeeded. No cancel, invoice, or refund handler.
GITHUB_TOKEN plus GITHUB_ORG or GITHUB_SITES_REPO	Yes, for the archive	Repo must already exist. Token needs write. An empty repo is bootstrapped once.
DEPLOYMENT_TOKEN	Yes, for managed hosting	Vercel token that can create production deployments.
DEPLOYMENT_TEAM_ID	If the token is a team token	Otherwise deploys land on the token's personal account.
PLATFORM_DOMAIN	Not used	Leave empty. The live URL is the Vercel-assigned host.
Database	Yes	Payment, Project, and Deployment rows are the source of truth, not GitHub.
Outbound HTTPS from the app	Yes	To Stripe, GitHub, api.vercel.com, and optionally HubSpot and Resend.

> There is no GitHub Action, no Vercel Git integration, and no DNS step in this pipeline. Connecting the GitHub repo to a Vercel project would be a second, unused path. The app already creates the Vercel project by posting a deployment named after the customer slug.

5. After the site is live

- Editing a paid site updates GitHub only. Vercel does not rebuild from that commit.
- The live site changes only when someone hits Update live site (POST /api/projects/{id}/deploy again). That uploads a new production deployment to the same Vercel project name.
- Cancelling or failing the Stripe subscription does not take the site down. The webhook does not listen for customer.subscription.deleted or failed invoices.
- Custom domain helpers exist in code (assignProjectDomain, aliasDeployment) and are not called. {slug}.{PLATFORM_DOMAIN} is not assigned. The architecture document's example (bright-smile.yourdomain.com) is the intended later shape, not what ships today.

6. Failure cases

- Webhook cannot reach the app: the delivery page still marks the project paid and syncs GitHub. Vercel still runs, because the page calls deploy itself. If the customer never opens that page and the webhook publish throws, the project stays PAID with deployment status FAILED until they open Deliver or someone retries the deploy endpoint.
- GitHub 404: repo missing or the token cannot see it. Sync status FAILED. Vercel can still succeed.
- Vercel 401 or 403: bad token or wrong team id. The project returns to PAID.

- Vercel is not READY within 90 seconds: treated as failure, same rollback to PAID.
- Double delivery: the webhook and the success page can both run. Payment is idempotent. Deploy is skipped if status is already LIVE or DEPLOYING.

7. Status values

Project

DRAFT, GENERATING, GENERATED, QUOTED, PAID, EXPORTED, DEPLOYING, LIVE, FAILED.

Paid, for GitHub re-sync, means PAID, EXPORTED, DEPLOYING, or LIVE.

Deployment

PENDING, REPOSITORY_CREATED, BUILD_STARTED, DEPLOYED, FAILED, CANCELLED.

The publish path writes BUILD_STARTED, then DEPLOYED or FAILED. REPOSITORY_CREATED is defined and not written by the current Vercel upload.

GitHub sync

PENDING, SYNCED, FAILED.

Source of this briefing: the application code as of 12 October 2026 (Stripe webhook, fulfillment, GitHub sync, and lib/deployment). Where architecture.md describes a custom subdomain or a Git-triggered deploy, the code does not do that yet.